Documentação da API Java

Equipe Alpha4
Sistema de gamificação para a Soul Up
Entrega da Sprint 4 - Domain Driven Design Using Java

Integrantes

Nome completo	RM
Vinicius de Oliveira	561862
Robison Custodio Junior	572866
Lucas Fernando Silva	564848
Thiago Shimizu Sodré dos Santos	572867
Arthur Mendes Garcia	571703

Turma: 1TDSPW - FIAP, São Paulo
Stack: Java 17, Spring Boot 3.5.16, JDBC, H2/PostgreSQL e Maven Wrapper.

Código e serviços publicados

Repositório Java: github.com/proc0x41/challengesprints-java.
Frontend: github.com/proc0x41/chellengesprints-alpha4.

API no Render: https://challengesprints-java.onrender.com/api.
Frontend na Vercel: https://chellengesprints-alpha4.vercel.app/.

As capturas desta documentação são reais e foram produzidas com o frontend consumindo o backend local. As pranchas de imagens e diagramas usam páginas ampliadas para manter textos e relações legíveis.

Objetivo, escopo e arquitetura

A Alpha4 propõe transformar atitudes sustentáveis em uma experiência recorrente por meio de missões, evidências, pontuação e participação coletiva. A API conecta as telas React ao armazenamento relacional e concentra as regras de negócio.

Funcionalidades implementadas

Missões: listagem, detalhe, criação, edição e exclusão administrativa; estado diário calculado por participante.

Perfis: criação de pseudônimo e sessão, leitura, edição e exclusão do próprio perfil; UF e comunidade opcional.

Evidências: envio textual, consulta própria, fila administrativa e aprovação ou rejeição explícita.

Ranking: pontuação persistida nas abrangências global, estadual, comunidade e amigos, por temporada bimestral.

Acompanhamentos: inclusão e remoção reais de participantes na lista de amigos, sem amizade fictícia.

Camadas e padrões utilizados

Resource - BO - DAO - banco: controladores definem REST; BOs aplicam validações, autorização e transações; DAOs executam SQL parametrizado. ConnectionFactory centraliza o DataSource via JdbcTemplate, que gerencia conexões em conjunto com as transações Spring.

CorsFilter aceita apenas origens configuradas. Sessões usam tokens aleatórios e somente hashes SHA-256 são persistidos. CRUD de missões e moderação exigem chave administrativa separada. JSON usa tipos estritos, sem converter strings ou números fracionários silenciosamente.

Limites assumidos de forma explícita

Guerra de Comunidades, recompensas e equivalências científicas de CO2 permanecem na proposta da Sprint 3 e não são funcionalidades entregues por esta API. As três missões iniciais são exemplos educacionais com pontos demonstrativos; não existem perfis fictícios no seed. A entrega de IA é independente.

Endpoints REST (1/2)

Base publicada: **https://challengesprints-java.onrender.com/api**.
Base local: **http://localhost:8080/api**. Respostas JSON UTF-8. Contrato completo em docs/CONTRATO_API.md.

Método	URI após /api	Acesso	OK	Falhas
GET	/health	Pública	200	500
POST	/participantes	Pública	201	400
GET	/participantes/me	Bearer	200	401
PUT	/participantes/me	Bearer	200	400, 401
DELETE	/participantes/me	Bearer	204	401
GET	/missoes	Bearer opcional	200	401
GET	/missoes/{id}	Bearer opcional	200	401, 404
POST	/missoes	Administrativa	201	400, 403, 503
PUT	/missoes/{id}	Administrativa	200	400, 403, 404, 503
DELETE	/missoes/{id}	Administrativa	204	403, 404, 409, 503
POST	/missoes/{id}/evidencias	Bearer	201	400, 401, 404, 409

Bearer: token da sessão do participante. **Administrativa:** cabeçalho X-Admin-Key igual a ADMIN_KEY. Chave administrativa ausente na configuração desabilita essas operações com 503. Nunca enviá-la ao bundle frontend.

Toda operação também pode retornar 500 em indisponibilidade interna ou do banco. Recursos inexistentes retornam 404; método não permitido, 405; formato não JSON, 415. Erros contêm somente a propriedade message, sem SQL, tokens ou stacktrace.

Endpoints REST (2/2)

Base publicada: **https://challengesprints-java.onrender.com/api**.
Base local: **http://localhost:8080/api**. Respostas JSON UTF-8. Contrato completo em docs/CONTRATO_API.md.

Método	URI após /api	Acesso	OK	Falhas
GET	/evidencias/me	Bearer	200	401
GET	/admin/evidencias	Administrativa	200	403, 503
PUT	/admin/evidencias/{id}	Administrativa	200	400, 403, 404, 409, 503
GET	/ranking?scope=global	Bearer opcional	200	400, 401
GET	/ranking?scope=estadual/comunidade/amigos	Bearer	200	400, 401
GET	/amigos	Bearer	200	401
POST	/amigos/{id}	Bearer	201	400, 401, 404, 409
DELETE	/amigos/{id}	Bearer	204	401, 404

Bearer: token da sessão do participante. **Administrativa:** cabeçalho X-Admin-Key igual a ADMIN_KEY. Chave administrativa ausente na configuração desabilita essas operações com 503. Nunca enviá-la ao bundle frontend.

Toda operação também pode retornar 500 em indisponibilidade interna ou do banco. Recursos inexistentes retornam 404; método não permitido, 405; formato não JSON, 415. Erros contêm somente a propriedade message, sem SQL, tokens ou stacktrace.

Contratos e regras de negócio

Exemplo de missão retornada

```
{"id":"uuid", "title":"Separar recicláveis",
 "description":"Orientações da missão",
 "category":"ambiental", "points":20,
 "requiresEvidence":true, "status":"disponivel"}
```

Categoria: ambiental, social ou governanca. Estado: disponivel, em-analise ou concluida. points é inteiro de 1 a 10.000; requiresEvidence é booleano. POST/PUT recebem os campos de conteúdo, sem id ou status. Tipos incompatíveis são rejeitados com 400.

Perfil e autenticação

POST /participantes recebe name, uf e community opcional e retorna participant e token. O pseudônimo possui de 2 a 80 caracteres após trim; a comunidade aceita até 80 e é normalizada para string vazia se omitida. A UF é uma sigla brasileira válida. O perfil não exige dados pessoais ou senha e não possui recuperação por e-mail.

Ciclo de evidência e crédito

POST /missoes/{id}/evidencias recebe text de 10 a 2.000 caracteres. O envio fica em análise e não credita pontos. Há uma evidência por missão, participante e dia civil em America/Sao_Paulo. Uma rejeitada pode ser corrigida; em análise ou aprovada, novo envio no mesmo dia retorna 409.

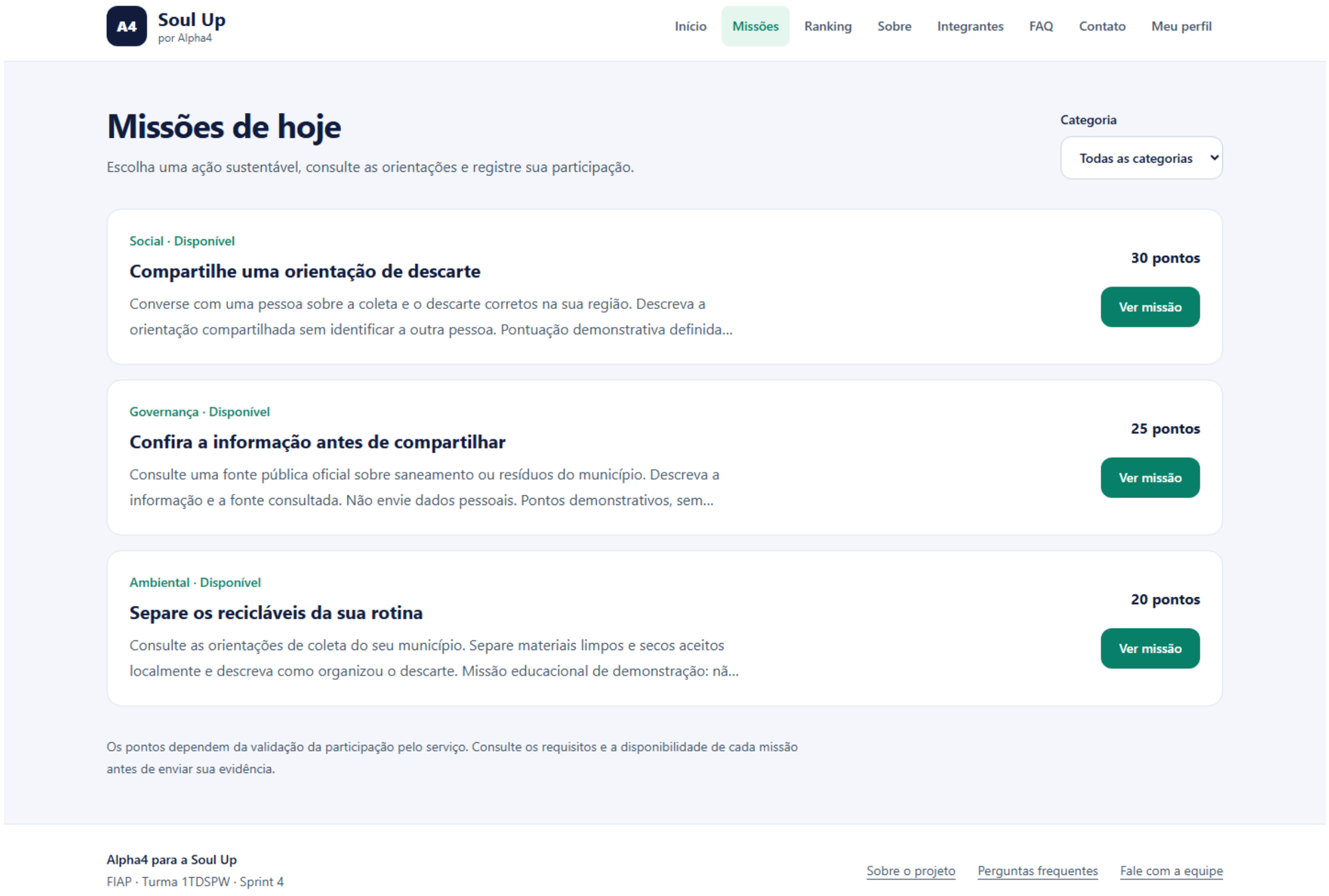
Os pontos da missão são registrados no envio (points_snapshot). A aprovação administrativa utiliza esse valor e é transacional: requisições simultâneas não duplicam o crédito. Alterar a missão não altera o valor já prometido a evidências anteriores. Missão com histórico não pode ser excluída (409).

Temporadas e segmentação

O ranking soma apenas evidências aprovadas cujo dia pertence ao bimestre corrente. Empates são ordenados por pseudônimo e ID. Estadual usa a UF do perfil; comunidade compara o nome sem diferenciar maiúsculas e retorna vazio para perfil sem grupo; amigos inclui o próprio perfil e os participantes efetivamente acompanhados.

Tela real: listagem de missões

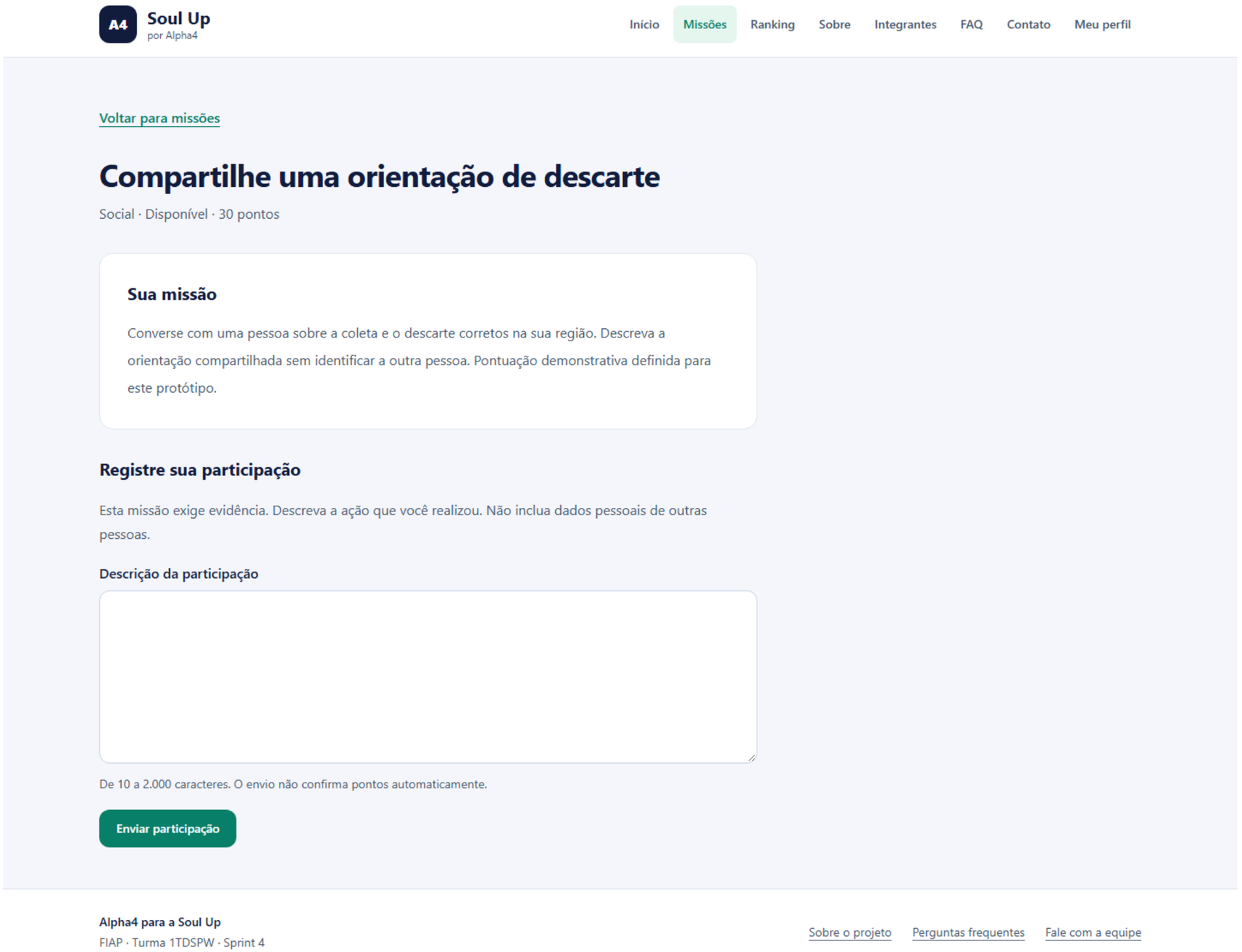
GET /api/missoes fornece os títulos, orientações, categorias e pontos exibidos. O filtro por categoria é local à interface e não substitui a fonte persistida. Nenhum erro de conexão é apresentado como lista fictícia.



Fonte: captura real do frontend integrado, 03/11/2026. Arquivo: docs/screenshots/missoes-desktop.png.

Tela real: detalhe e envio de evidência

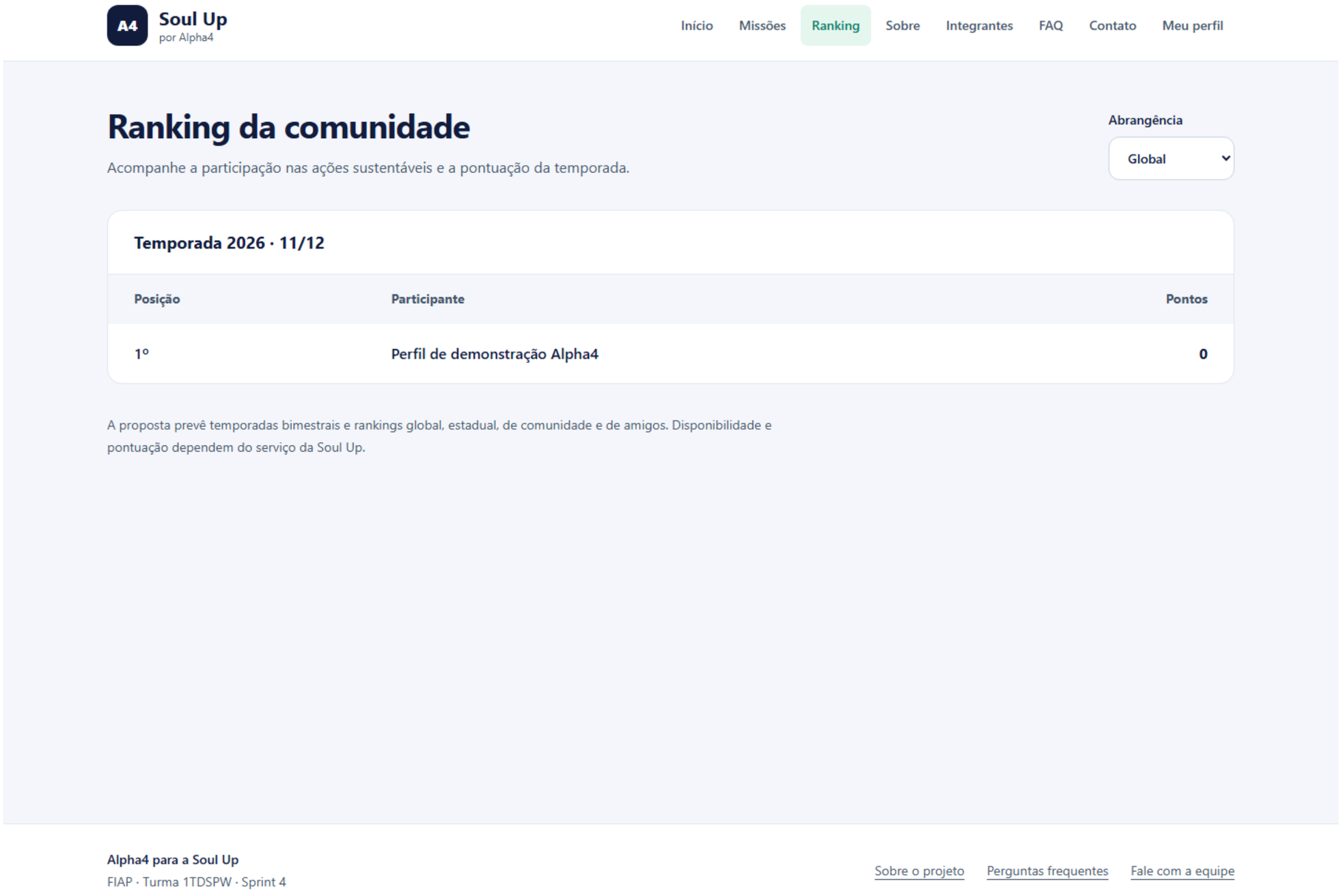
A rota dinâmica /missoes/:id consulta GET /api/missoes/{id}. O formulário envia POST /api/missoes/{id}/evidencias com a sessão Bearer. O backend valida o texto e mantém a participação em análise até a moderação explícita.



Fonte: captura real do frontend integrado, 03/11/2026. Arquivo: docs/screenshots/detalhe-desktop.png.

Tela real: ranking integrado

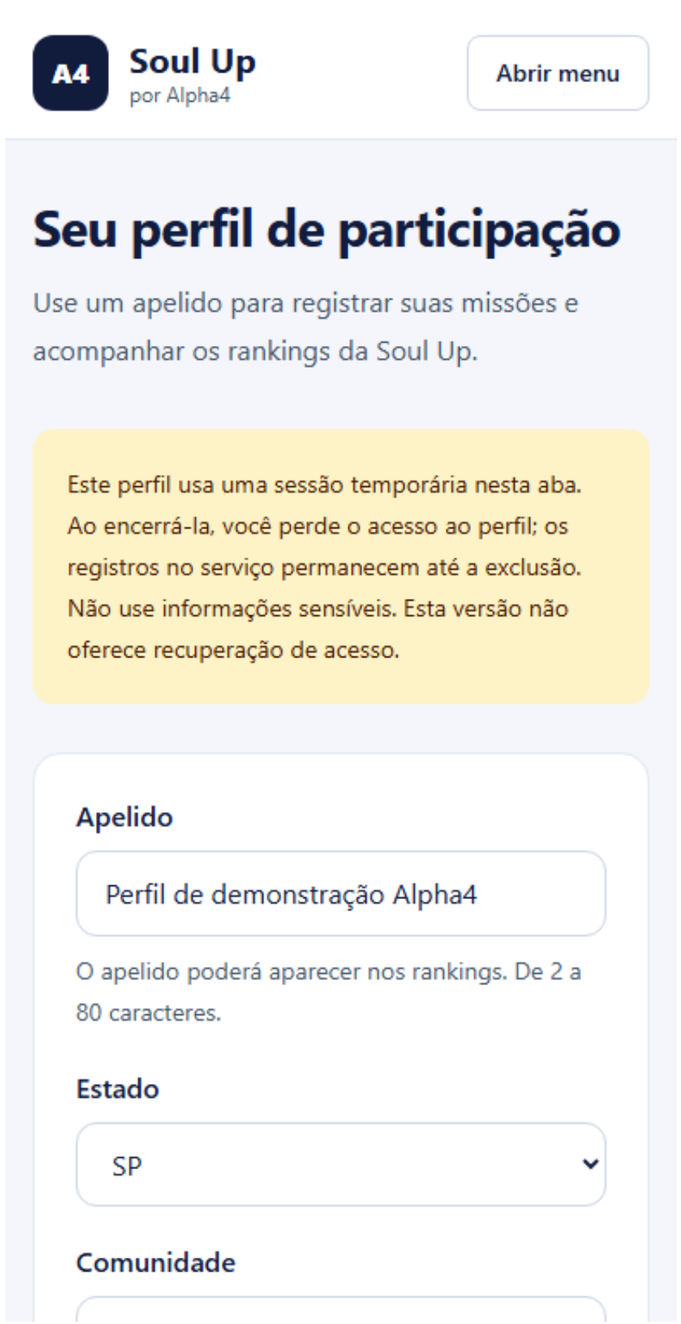
GET /api/ranking retorna temporada, posições e pontuação. As posições mostradas são registros reais do ambiente local de demonstração, não resultados ambientais mensurados. Os escopos pessoais usam a sessão do participante.



Fonte: captura real do frontend integrado, 03/11/2026. Arquivo: docs/screenshots/ranking-desktop.png.

Tela real: perfil em dispositivo móvel

Captura completa do perfil responsivo em 390 px, dividida em três faixas com pequena sobreposição para preservar linhas nas transições. Ler da esquerda para a direita. O perfil permite criar/editar a sessão e acompanhar participantes reais; sem chave administrativa no navegador.



Parte 1/3



Parte 2/3

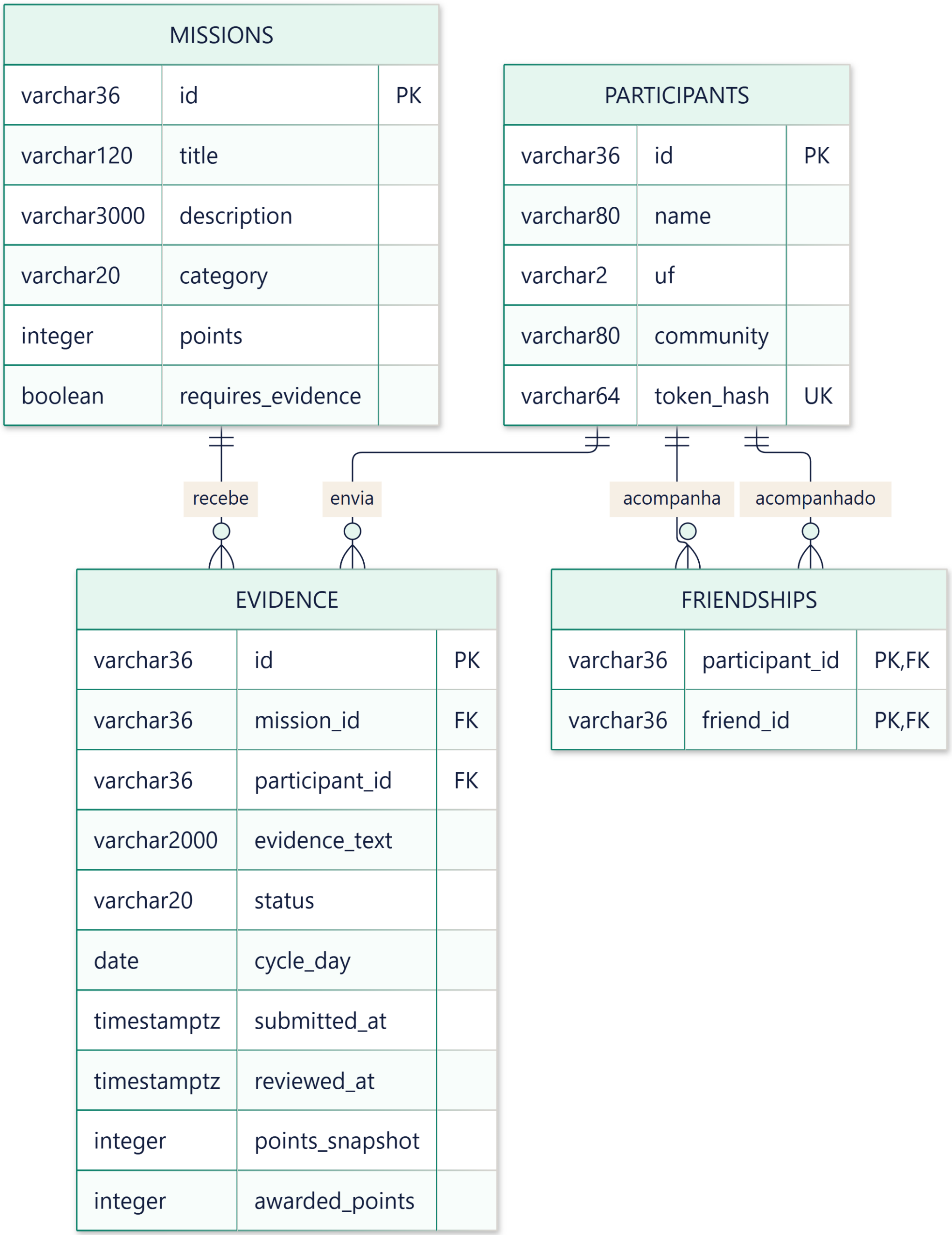


Parte 3/3

Fonte: docs/screenshots/perfil-mobile.png. Captura real contínua do frontend integrado, 03/11/2026.

Modelo de Entidade-Relacionamento

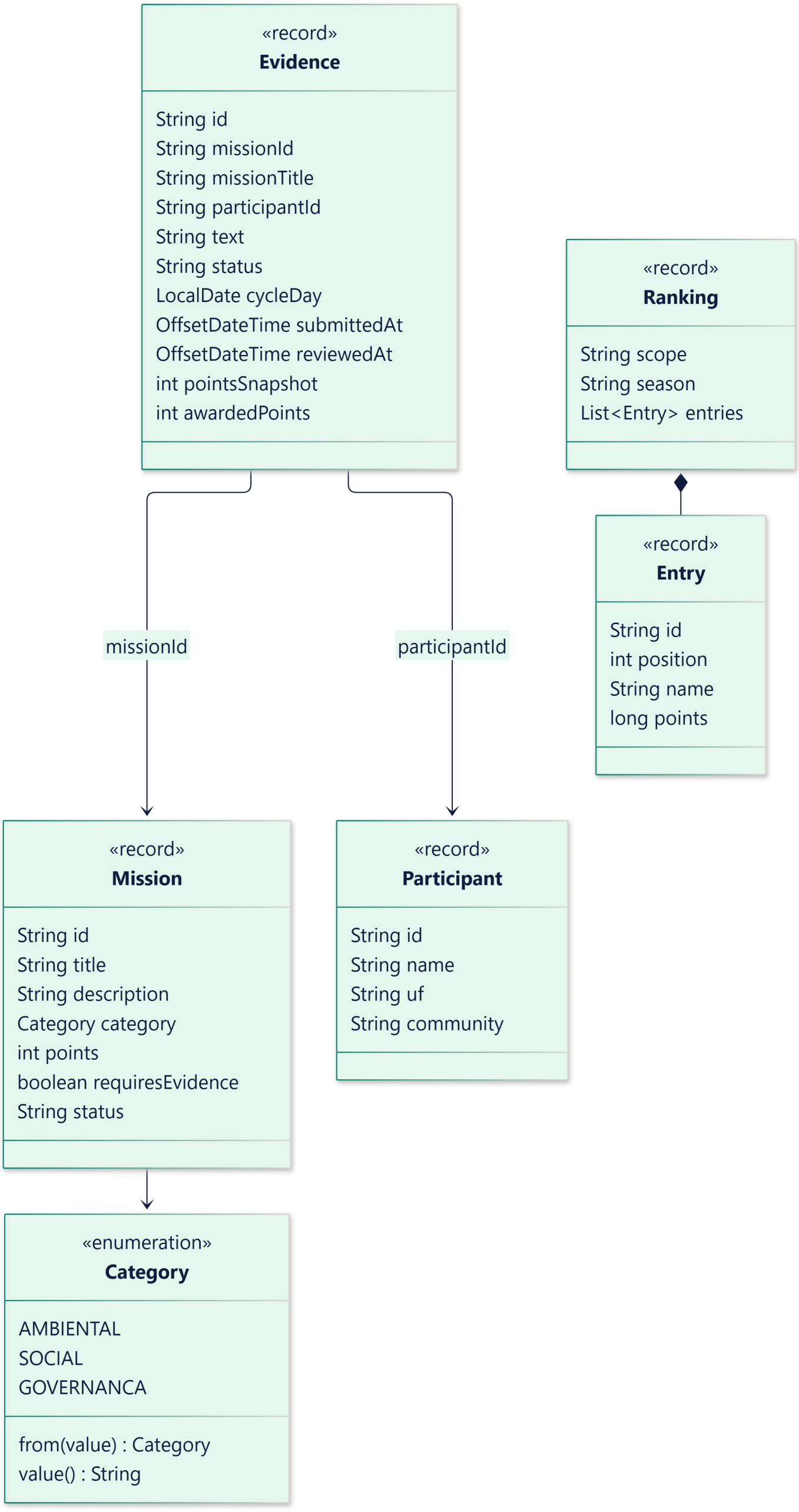
Modelo compatível com src/main/resources/schema.sql. IDs são chaves primárias; evidências referenciam missão e participante. O trio missão/participante/dia é único. Acompanhamentos possuem chave composta e impedem autoacompanhamento. O token é persistido somente por hash.



Fonte versionada: docs/mer.svg. Prancha ampliada para preservar a legibilidade.

Classes: modelos de domínio

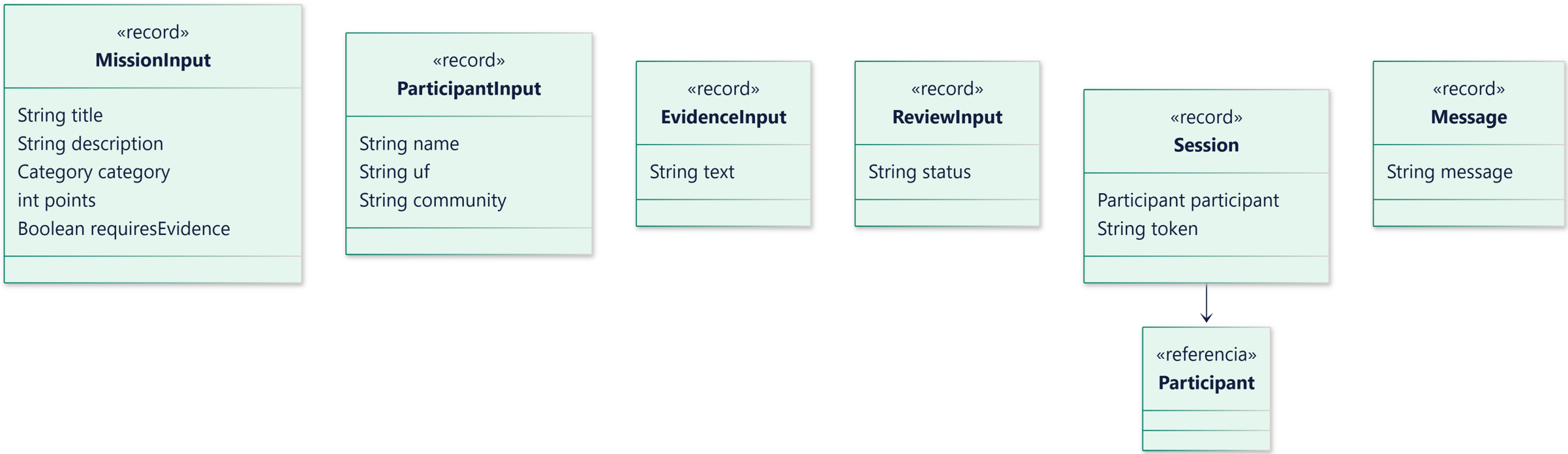
Records imutáveis representam os dados do domínio e do ranking. Category restringe categorias. Os identificadores de Evidence referenciam Mission e Participant; Ranking contém suas entradas.



Fonte versionada: docs/diagrams/classes-01-dominio.svg. Prancha ampliada para preservar a legibilidade.

Classes: contratos de entrada e saída

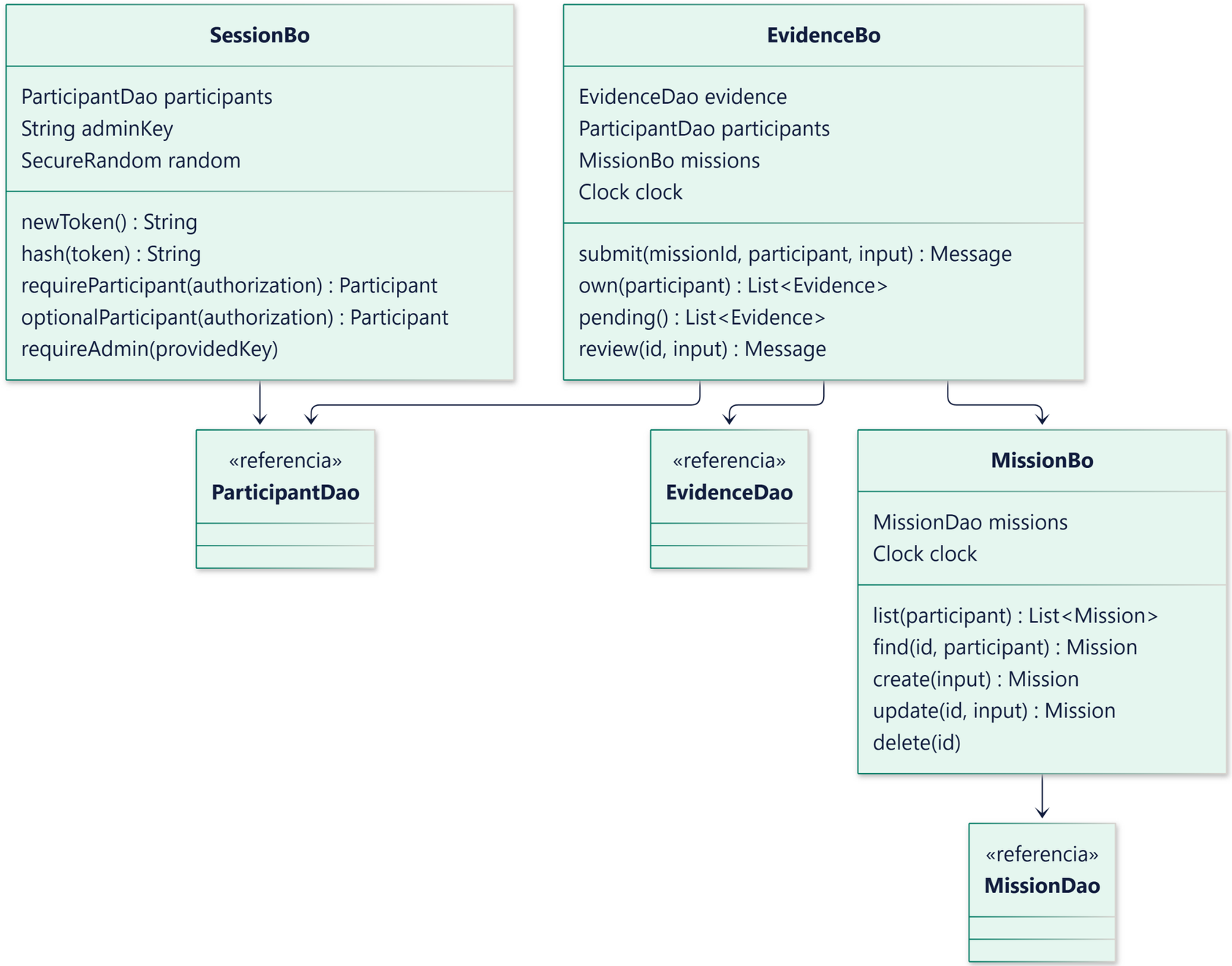
DTOs declarados em Inputs.java separam os dados aceitos pela API dos modelos persistidos. Validações Jakarta e BOs impõem campos, tipos e limites. Session contém o perfil e o token entregue uma única vez na criação.



Fonte versionada: docs/diagrams/classes-02-dtos.svg. Prancha ampliada para preservar a legibilidade.

Classes: regras de missões e evidências

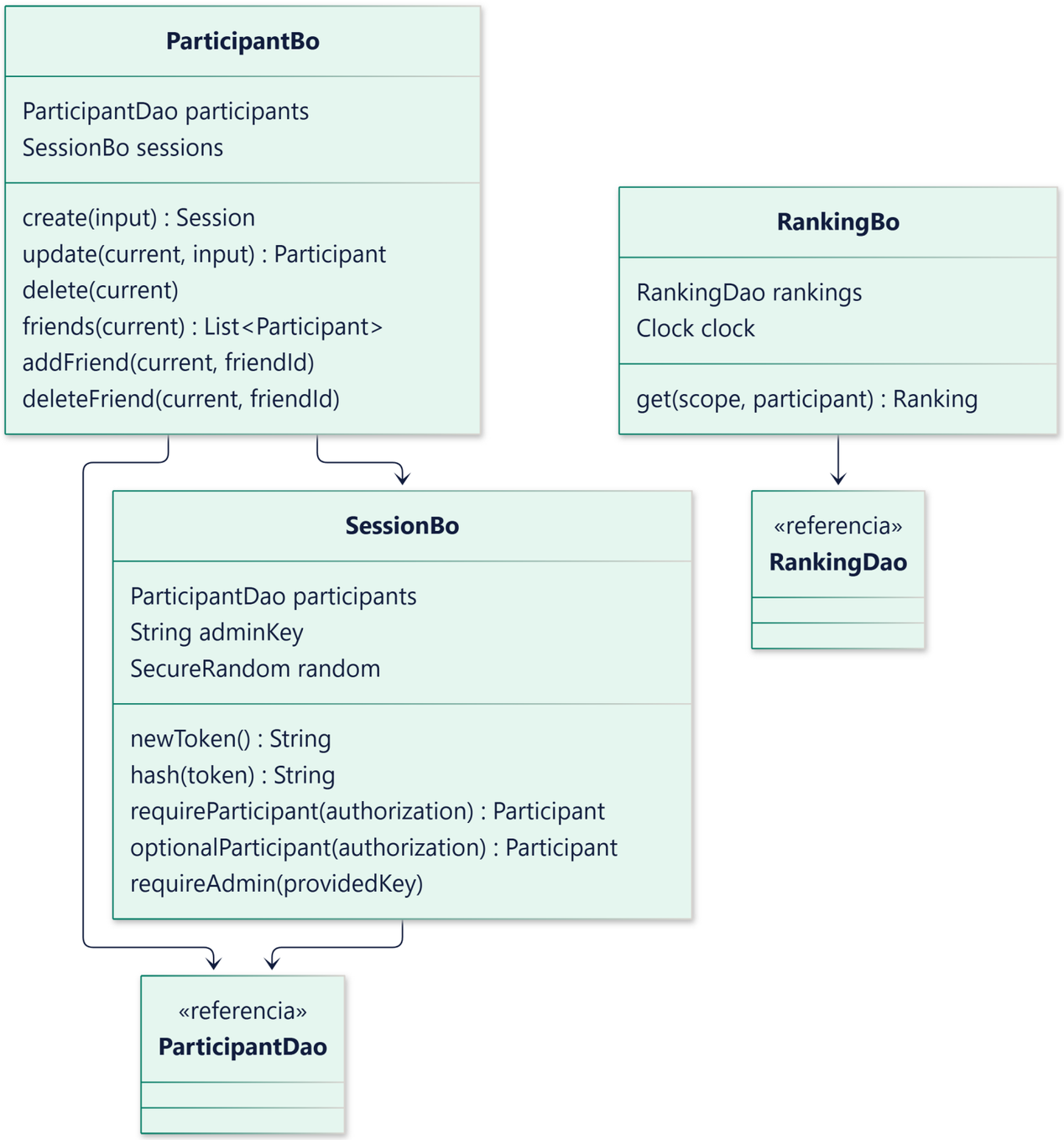
MissionBo concentra o CRUD; EvidenceBo organiza envio diário e revisão em transação. SessionBo autentica participantes e autoriza administração. Caixas marcadas como referência têm seus detalhes nas pranchas da própria camada.



Fonte versionada: docs/diagrams/classes-03-regras-missoes.svg. Prancha ampliada para preservar a legibilidade.

Classes: perfis e temporadas

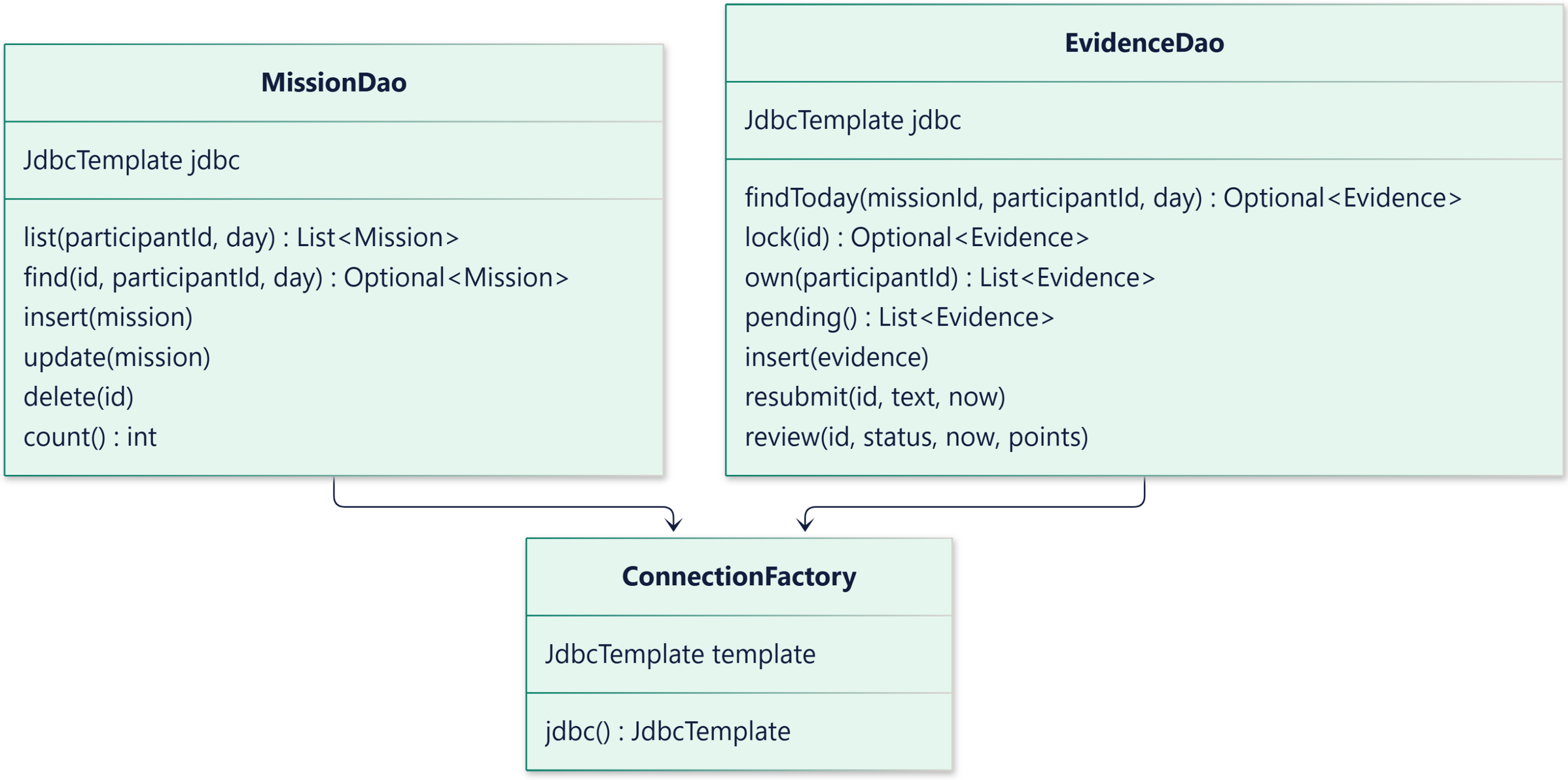
ParticipantBo normaliza perfis e mantém acompanhamentos; RankingBo delimita abrangência e bimestre a partir do Clock em America/Sao_Paulo. As consultas ficam nos respectivos DAOs.



Fonte versionada: docs/diagrams/classes-04-regras-perfis.svg. Prancha ampliada para preservar a legibilidade.

Classes: persistência de missões e evidências

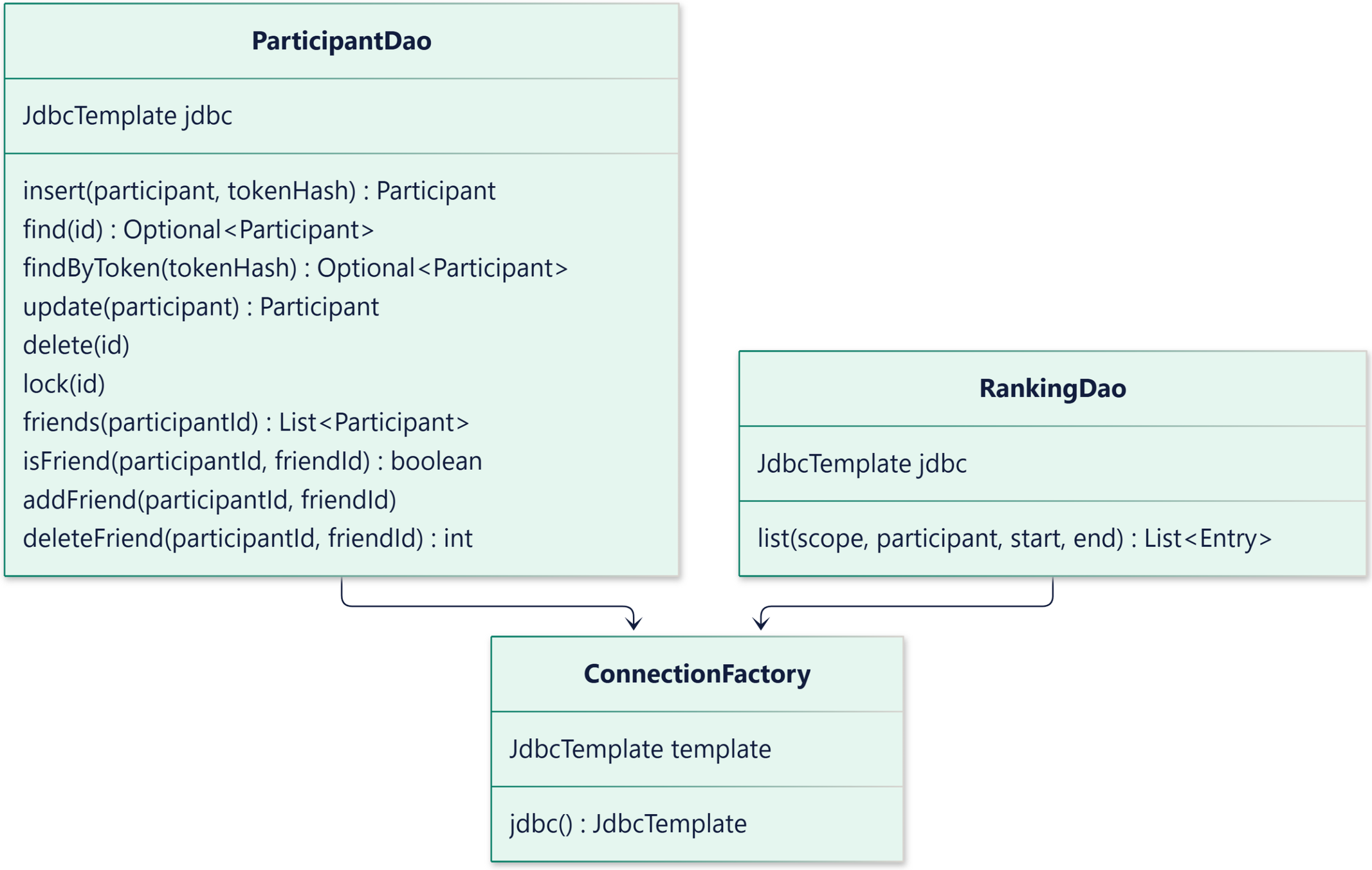
Os DAOs executam SQL parametrizado e mapeiam registros. EvidenceDao usa bloqueio no processo de revisão; MissionDao calcula o estado diário individual. ConnectionFactory fornece o JdbcTemplate compartilhado.



Fonte versionada: docs/diagrams/classes-05-dao-missoes.svg. Prancha ampliada para preservar a legibilidade.

Classes: persistência de perfis e ranking

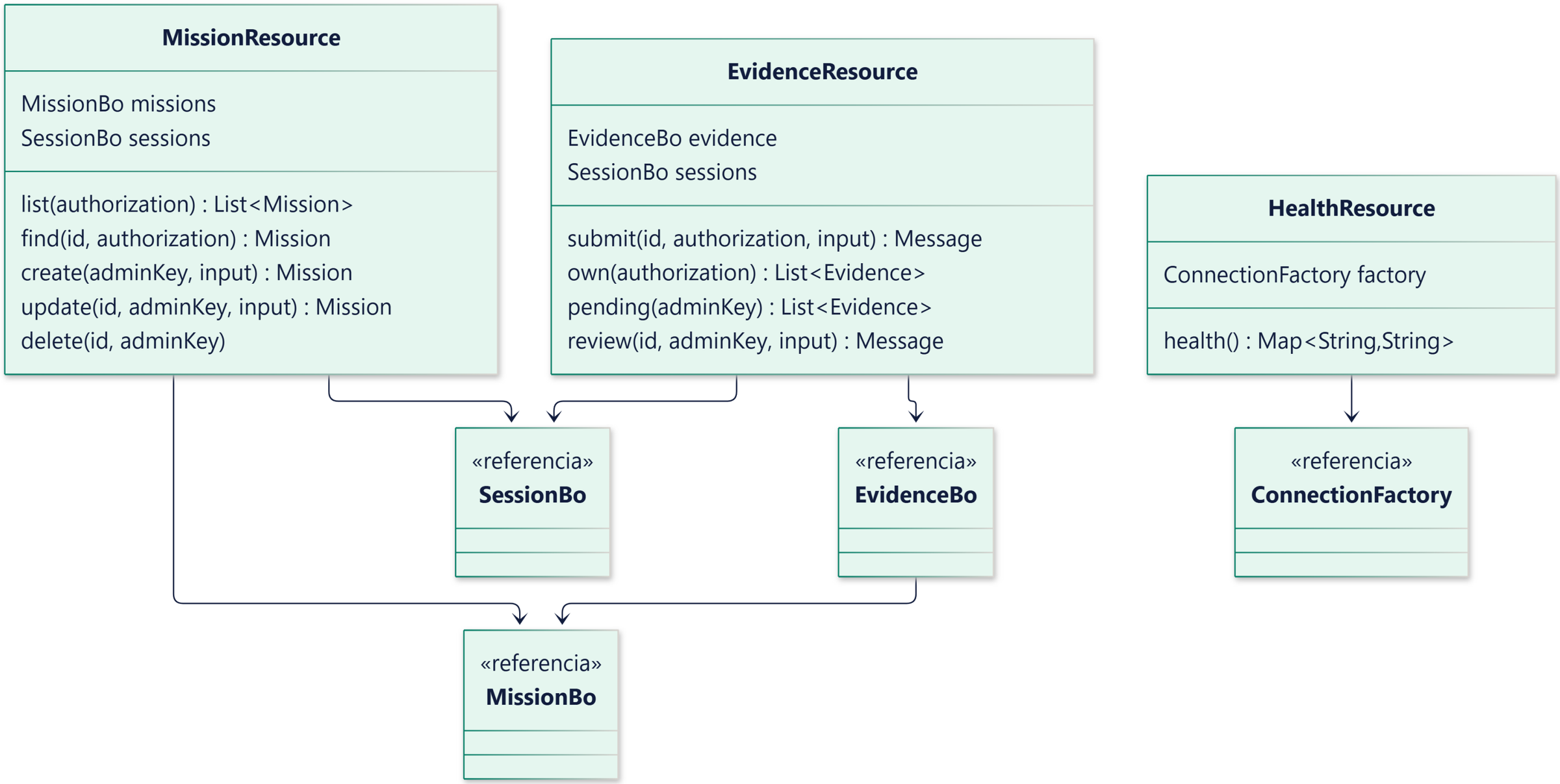
ParticipantDao mantém perfis e relações reais. RankingDao filtra apenas escopos permitidos e soma pontos aprovados da temporada. A conexão é centralizada, sem SQL espalhado pelos controladores.



Fonte versionada: docs/diagrams/classes-06-dao-perfis.svg. Prancha ampliada para preservar a legibilidade.

Classes: recursos REST de missões

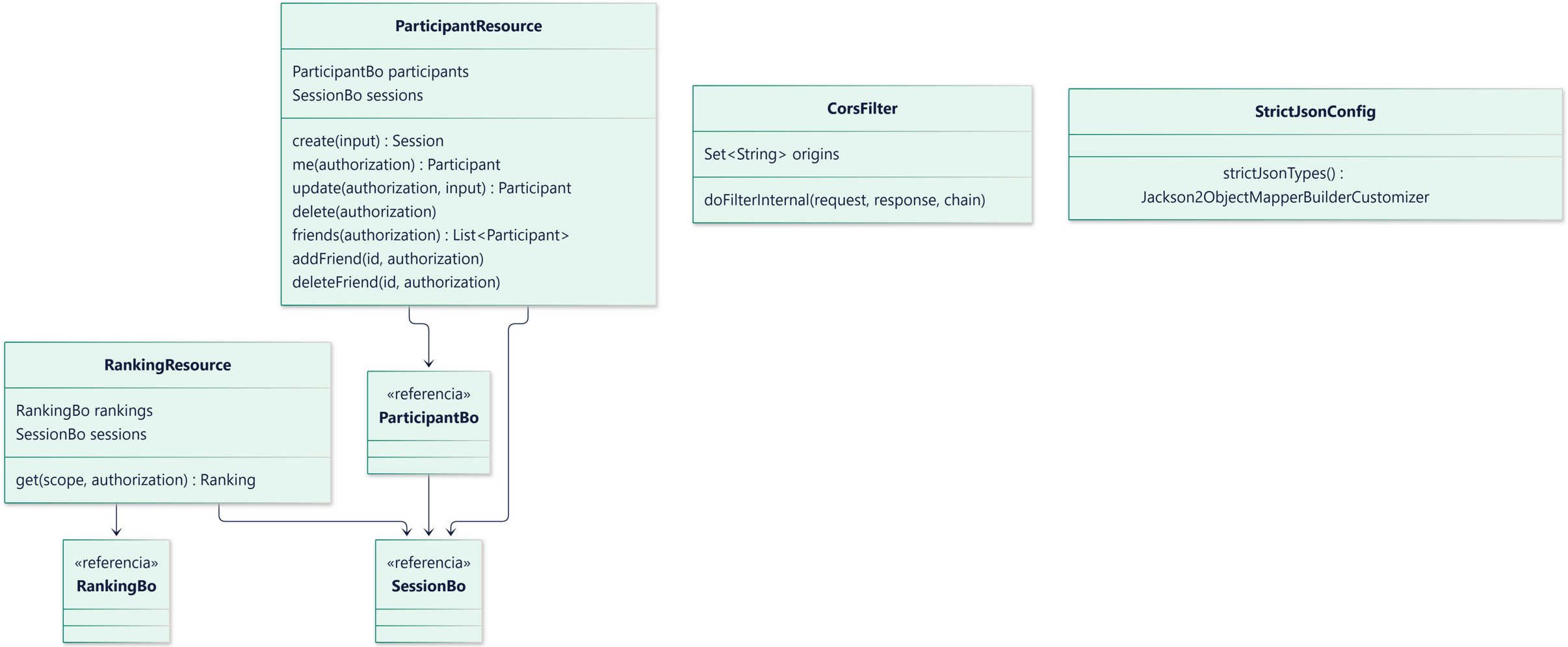
MissionResource e EvidenceResource traduzem HTTP para as operações dos BOs. HealthResource verifica a conexão com o banco por SELECT 1. A autorização é aplicada antes das operações administrativas.



Fonte versionada: docs/diagrams/classes-07-recursos-missoes.svg. Prancha ampliada para preservar a legibilidade.

Classes: recursos de perfil e configuração

ParticipantResource e RankingResource expõem operações pessoais e públicas. CorsFilter restringe origens; StrictJsonConfig impede coerções silenciosas de tipos. A fonte canônica completa permanece em docs/classes.mmd.



Fonte versionada: docs/diagrams/classes-08-recursos-perfis.svg. Prancha ampliada para preservar a legibilidade.

Procedimentos de execução

Ferramentas e requisitos

JDK 17 ou superior (validado com Microsoft OpenJDK 17.0.18), terminal e rede na primeira instalação. O Maven Wrapper baixa Maven 3.9.11 com verificação da distribuição. Spring Boot 3.5.16 está fixado em pom.xml. IDEs opcionais: IntelliJ, Eclipse ou VS Code com suporte a Java.

Passo a passo no Windows PowerShell

```
git clone https://github.com/proc0x41/challengesprints-java

cd challengesprints-java

java -version

.\mvnw.cmd clean verify

.\mvnw.cmd spring-boot:run
```

Para importar numa IDE, abra pom.xml como projeto Maven, selecione JDK 17 e aguarde a resolução das dependências. Execute SoulUpApplication. Em Linux/macOS, use sh mvnw nos mesmos objetivos Maven.

Verificar o serviço e consumir pelo frontend

```
Invoke-RestMethod http://localhost:8080/api/health

Invoke-RestMethod http://localhost:8080/api/missoes
```

No frontend local, configure VITE_API_URL=http://localhost:8080/api. Na Vercel, use VITE_API_URL=https://challengesprints-java.onrender.com/api e gere novo deploy. O backend permite localhost/127.0.0.1:5173 e a origem publicada definida por FRONTEND_ORIGIN, cujo padrão é https://chellengesprints-alpha4.vercel.app. CORS_ORIGINS configura origens adicionais.

Executar o artefato empacotado

```
.\mvnw.cmd package

java -jar target/soul-up-api-0.1.2.jar
```

O banco H2 fica em data/soulup.mv.db e persiste entre reinicializações. Os testes usam outro banco, isolado em memória, e não alteram o banco do ambiente de demonstração.

Configuração, verificação e entrega

Variável de ambiente	Finalidade
PORT	Porta HTTP; padrão 8080.
ADMIN_KEY	Chave aleatória para CRUD e moderação; vazia resulta em 503. Nunca VITE_*.
CORS_ORIGINS	Origens exatas permitidas, separadas por vírgula.
FRONTEND_ORIGIN	Origem publicada; padrão https://chellengesprints-alpha4.vercel.app.
JDBC_DATABASE_URL	URL JDBC H2 local ou PostgreSQL para nuvem.
DB_USER / DB_PASSWORD	Credenciais do banco, configuradas fora do repositório.
SEED_DEMO	Habilita exemplos educacionais somente se a tabela estiver vazia.

O arquivo .env.example é uma referência; Spring Boot não o lê automaticamente. Exporte as variáveis no shell ou configure-as no serviço. Para container, use o Dockerfile fornecido. Em nuvem, configure PostgreSQL persistente; H2 sem volume não preserva dados ao substituir o container.

Evidências de verificação

11 testes de integração passaram, sem falhas, erros ou testes ignorados: CRUD, validações de tipos estritos, limites após trim, hash do token, correção de evidência, snapshot de pontos, concorrência no envio e na aprovação, meia-noite de São Paulo, temporadas e CORS. Relatório reproduzível: .\mvnw.cmd test.

A revisão independente encontrou coerção indevida de JSON e divergência nos limites antes do trim; ambas foram corrigidas e cobertas por regressões HTTP. A integração real do frontend foi exercitada com persistência e moderação explícita. As capturas deste PDF documentam essas telas locais.

Conteúdo da entrega

O ZIP de fontes inclui código Java, pom.xml, Maven Wrapper, configurações, contratos, fontes/renderizações dos diagramas, capturas reais, script desta documentação e este PDF. Exclui .git, target, banco data, variáveis locais e tokens.

API e frontend possuem as URLs públicas indicadas na capa. As próximas páginas apresentam a configuração PostgreSQL/Render incluída no projeto e o resultado verificável dos testes HTTP. Capturas locais e respostas remotas estão identificadas separadamente.

Produção: Render e PostgreSQL

Serviço já publicado: manter a URL atual

API: https://challengesprints-java.onrender.com/api. Frontend: https://chellengesprints-alpha4.vercel.app/.

Crie o PostgreSQL na mesma região da API. No serviço Java existente, configure os valores reais de host interno, banco e credenciais fornecidos pelo painel. O perfil padrão aceita a URL JDBC abaixo; nenhum segredo deve ser versionado.

```
JDBC_DATABASE_URL=jdbc:postgresql://HOST_INTERNO:5432/BANCO?sslmode=require

DB_USER=USUARIO_DO_BANCO

DB_PASSWORD=SENHA_DO_BANCO

CORS_ORIGINS=https://chellengesprints-alpha4.vercel.app
```

ADMIN_KEY deve ser longa e aleatória para CRUD de missões e moderação. Após salvar as variáveis, faça redeploy. O schema.sql inicializa tabelas e restrições no banco configurado, e GET /api/health verifica a conexão com SELECT 1.

Instalação automatizada incluída no ZIP

O render.yaml declara web service Docker e PostgreSQL 17 no Render, com plano free, rede privada e credenciais referenciadas por fromDatabase. O perfil render recebe DB_HOST, DB_PORT, DB_NAME, DB_USER e DB_PASSWORD e constrói a URL JDBC com TLS. ADMIN_KEY é gerada pelo Blueprint. Para instalação nova, use New / Blueprint e selecione o repositório.

Para preservar o serviço existente e sua URL, use as variáveis da seção anterior. PostgreSQL free expira após 30 dias; verificar as condições atuais do provedor antes de uso continuado.

Referências: render.com/docs/blueprint-spec; render.com/docs/postgresql-creating-connecting; render.com/docs/free.

Verificações e testes da aplicação

Consultas GET públicas

Registro: **2026-11-04T23:32:20.715080-03:00**. Fonte: docs/evidencias/deploy.json, produzida por python scripts/check_deploy.py. As consultas abaixo verificam a disponibilidade dos recursos públicos.

Verificação	Método / recurso	Resposta
Saúde e consulta ao banco	GET /health	200
Listagem pública	GET /missoes	200

CORS e regras de negócio nos testes de integração

Os 11 testes de integração passaram. A origem da Vercel recebe preflight 204 e GET 200 no ambiente de teste, inclusive quando CORS_ORIGINS contém apenas localhost. O domínio publicado é configurado por FRONTEND_ORIGIN; origens não listadas e domínios com sufixos semelhantes recebem 403. A suíte também exercita CRUD, autorização, evidências, pontuação e temporadas.

Reprodução: mvnw.cmd test. O banco desses testes é H2 isolado em memória. O contrato REST e os códigos de resposta estão nas tabelas de endpoints e em docs/CONTRATO_API.md.

Execução com PostgreSQL Docker

Verificação local isolada em 2026-11-04T23:31:22.908772-03:00: PostgreSQL 17.11 e API 0.1.2, perfil render. Criação/atualização de perfil, evidência, aprovação, ranking e exclusão passaram. Perfil, token e pontos continuaram disponíveis após reiniciar a API. Relatório: docs/evidencias/postgresql-local.json.

TLS foi desabilitado somente nessa conexão Docker local; o Blueprint Render mantém sslmode=require. O ZIP inclui os relatórios e scripts de reprodução, sem credenciais. Para executar novamente: python scripts/check_postgres.py.